

Sprawozdanie z projektu programistycznego

Strava Clone

Aplikacja do śledzenia aktywności fizycznych

Projektowanie i Programowanie Systemów Internetowych I

Technologia backendu:	Laravel 12 (PHP 8.2+)
Technologia frontendu:	Vue 3 + TypeScript
Baza danych:	PostgreSQL 18
Repozytorium:	github.com/Console-dungeon/ strava-clone

19 maja 2026

Spis treści

1	Opis funkcjonalny systemu	2
1.1	Cel i zakres projektu	2
1.2	Moduły funkcjonalne	2
1.2.1	Rejestracja i uwierzytelnianie	2
1.2.2	Zarządzanie aktywnościami	2
1.2.3	Integracja z Garmin Connect	3
1.2.4	Dashboard i statystyki	3
1.2.5	Wizualizacja trasy na mapie	3
1.2.6	Logi aktywności użytkownika	3
1.2.7	Internacjonalizacja	4
1.2.8	Profil użytkownika	4
1.3	Diagram przepływu użytkownika	4
2	Opis technologiczny	5
2.1	Architektura ogólna	5
2.2	Backend	5
2.2.1	Laravel 12	5
2.2.2	Wzorzec Controller–Service	6
2.2.3	Inertia.js	6
2.2.4	Kluczowe serwisy	6
2.2.4.1	GarminService	6
2.2.4.2	GpxParser	6
2.2.4.3	DashboardService	7
2.2.4.4	ActivityLogger	7
2.2.5	Uwierzytelnianie	7
2.3	Frontend	7
2.3.1	Vue 3 + TypeScript	7
2.3.2	Kluczowe biblioteki frontendowe	7
2.3.3	Struktura plików frontendowych	8
2.3.4	Konwencje frontendowe	8

2.4	Środowisko deweloperskie	9
2.4.1	Docker / Laravel Sail	9
2.4.2	Dev Container	9
2.4.3	Haki pre-commit	9
3	Wdrożone zagadnienia kwalifikacyjne	10
3.1	Architektura i wzorce projektowe	10
3.2	Bezpieczeństwo	11
3.3	Integracje zewnętrzne	11
3.4	Frontend i UX	12
3.5	Jakość kodu i DevOps	12
4	Instrukcja uruchomienia systemu	14
4.1	Wymagania wstępne	14
4.1.1	Uruchomienie lokalne (Dev Container)	14
4.1.2	Uruchomienie zdalne / produkcyjne	14
4.2	Uruchomienie lokalne (Dev Container – zalecane)	14
4.2.1	Krok 1: Sklonowanie repozytorium	14
4.2.2	Krok 2: Inicjalizacja środowiska	14
4.2.3	Krok 3: Otwarcie w kontenerze	15
4.2.4	Krok 4: Migracja bazy danych	15
4.2.5	Krok 5: Uruchomienie serwera deweloperskiego	15
4.2.6	Alternatywne uruchomienie przez F5	15
4.3	Konfiguracja integracji Garmin (opcjonalne)	15
4.4	Uruchomienie zdalne / produkcyjne	16
4.4.1	Krok 1: Przygotowanie serwera	16
4.4.2	Krok 2: Sklonowanie i konfiguracja	16
4.4.3	Krok 3: Konfiguracja zmiennych środowiskowych	16
4.4.4	Krok 4: Instalacja zależności i build	16
4.4.5	Krok 5: Konfiguracja serwera WWW	17
4.4.6	Krok 6: Uruchomienie workera kolejki	17
4.4.7	Krok 7: Uprawnienia katalogów	18
4.5	Przydatne komendy	18
5	Wnioski projektowe	19
5.1	Podsumowanie realizacji projektu	19
5.2	Kluczowe wnioski	19
5.2.1	Bezpośrednia komunikacja zespołu jako fundament projektu	19
5.2.2	Iteracyjna praca z małymi przyrostami	20
5.2.3	Praca zespołowa jako akcelerator skali i jakości	20

5.2.4	Agenci AI jako katalizator produktywności	20
5.3	Rekomendacje na przyszłość	21
5.4	Zakończenie	21

Rozdział 1

Opis funkcjonalny systemu

1.1 Cel i zakres projektu

Strava Clone jest aplikacją webową przeznaczoną do rejestrowania, analizowania i wizualizacji aktywności fizycznych użytkownika. Stanowi odwzorowanie kluczowych funkcjonalności popularnej platformy Strava, z naciskiem na integrację z urządzeniami Garmin oraz wsparciem dla plików GPS.

Aplikacja adresowana jest do osób aktywnych fizycznie – biegaczy, rowerzystów i pływaków – którzy chcą mieć wgląd w historię swoich treningów i monitorować postępy w jednym miejscu.

1.2 Moduły funkcjonalne

1.2.1 Rejestracja i uwierzytelnianie

System obsługuje pełny cykl życia konta użytkownika:

- rejestracja nowego konta z walidacją adresu e-mail,
- logowanie i wylogowanie z obsługą sesji,
- reset hasła przez e-mail (token jednorazowy),
- weryfikacja adresu e-mail,
- zmiana hasła i danych profilowych,
- usunięcie konta z usunięciem wszystkich powiązanych danych.

Moduł oparty jest na pakiecie *Laravel Breeze* i nie wymaga modyfikacji.

1.2.2 Zarządzanie aktywnościami

Rdzeń funkcjonalności aplikacji umożliwia:

- ręczne dodanie aktywności (typ, dystans, czas, notatki),
- import aktywności z pliku GPX – aplikacja parsuje trasę i oblicza parametry,
- przeglądanie listy aktywności z filtrowaniem po typie i paginacją,
- wyświetlanie szczegółów aktywności wraz z interaktywną mapą trasy,
- usuwanie aktywności.

Obsługiwane typy aktywności: bieg (*running*), jazda na rowerze (*cycling*), pływanie (*swimming*) oraz inne.

1.2.3 Integracja z Garmin Connect

Aplikacja umożliwia:

- jednorazowy pełny import historii aktywności z konta Garmin Connect (do 1000 aktywności),
- synchronizację nowych aktywności (brakujące od ostatniego importu),
- pobranie szczegółowych danych aktywności Garmin: średnie i maksymalne tętno, spalone kalorie, prędkość maksymalna,
- przechowywanie danych uwierzytelniających Garmin zaszyfrowanych w bazie danych.

1.2.4 Dashboard i statystyki

Strona główna zalogowanego użytkownika prezentuje:

- karty podsumowania: łączny dystans, łączny czas, średnia prędkość,
- wykres liniowy aktywności w czasie (za pomocą biblioteki Unovis),
- tabelę ostatnich aktywności z najważniejszymi parametrami.

1.2.5 Wizualizacja trasy na mapie

Dla aktywności zaimportowanych z pliku GPX lub Garmina dostępna jest interaktywna mapa oparta na bibliotece Leaflet z OpenStreetMap, prezentująca:

- narysowaną trasę aktywności,
- marker punktu startowego i końcowego trasy.

1.2.6 Logi aktywności użytkownika

System rejestruje kluczowe akcje użytkownika w tabeli `activity_logs`:

- uruchomienie synchronizacji Garmin,
- zakończenie synchronizacji Garmin,
- utworzenie aktywności,

- usunięcie aktywności.

1.2.7 Internacjonalizacja

Interfejs dostępny jest w dwóch językach:

- **polskim** – domyślnym (zmienna `APP_LOCALE=pl`),
- **angielskim** – jako język zapasowy.

Tłumaczenia przechowywane są w plikach TypeScript `pl.ts` i `en.ts`.

1.2.8 Profil użytkownika

Moduł profilu pozwala użytkownikowi:

- zaktualizować imię i adres e-mail,
- zmienić hasło,
- podłączyć i zapisać dane logowania do konta Garmin Connect,
- trwale usunąć konto.

1.3 Diagram przepływu użytkownika

1. Nowy użytkownik rejestruje konto i weryfikuje e-mail.
2. Po zalogowaniu widzi pusty dashboard z zaproszeniem do dodania aktywności.
3. Użytkownik może ręcznie wpisać aktywność, wgrać plik GPX lub powiązać konto Garmin.
4. Po imporcie aktywności dashboard wyświetla statystyki i wykresy.
5. Kliknięcie aktywności otwiera szczegóły z mapą trasy (jeśli dostępna).
6. Regularne synchronizacje Garmin mogą być uruchamiane jednym kliknięciem.

Rozdział 2

Opis technologiczny

2.1 Architektura ogólna

Aplikacja zbudowana jest w architekturze monolitycznej z wbudowanym podziałem na warstwy, opartej na wzorcu MVC rozszerzonym o warstwę serwisów. Komunikacja między backendem a frontendem odbywa się przez protokół **Inertia.js**, który eliminuje potrzebę budowania osobnego REST API – backend zwraca komponenty Vue zamiast JSON.

Warstwa	Technologia
Backend framework	Laravel 12 (PHP 8.2+)
Frontend framework	Vue 3 + TypeScript
Protokół SPA	Inertia.js 2.0
Baza danych	PostgreSQL 18
Bundler	Vite 7
Serwer deweloperski	Laravel Sail (Docker)

2.2 Backend

2.2.1 Laravel 12

Framework PHP w wersji 12 dostarcza:

- **Routing** – deklaracyjny w `routes/web.php` i `routes/auth.php`,
- **Eloquent ORM** – mapowanie obiektowo-relacyjne dla PostgreSQL,
- **Form Requests** – walidacja danych wejściowych w osobnych klasach,
- **Queue** – asynchroniczne przetwarzanie zadań (driver: `database`),
- **Cache** – buforowanie danych (driver: `database`),
- **Szyfrowanie** – kolumna `garmin_password` przechowywana zaszyfrowana (`encrypted cast`).

2.2.2 Wzorzec Controller–Service

Kontrolery są celowo cienkie: nie zawierają logiki biznesowej, jedynie delegują pracę do warstwy serwisów.

```
1 class DashboardController extends Controller
2 {
3     public function __construct(
4         private readonly DashboardService $dashboard
5     ) {}
6
7     public function __invoke(Request $request): Response
8     {
9         return Inertia::render('Dashboard', [
10             'data' => $this->dashboard->getData(
11                 auth()->guard()->user()
12             ),
13         ]);
14     }
15 }
```

Listing 2.1: Przykładowy kontroler delegujący do serwisu

2.2.3 Inertia.js

Inertia.js pełni rolę warstwy pośredniczącej między backendem a frontendem. Backend nie zwraca JSON, lecz wywołuje `Inertia::render('NazwaStrony', $props)`, co powoduje załadowanie odpowiedniego komponentu Vue z danymi jako *props*. Nawigacja po aplikacji odbywa się bez przeładowania strony (SPA), przy czym serwer zawsze kontroluje dane i routowanie.

2.2.4 Kluczowe serwisy

2.2.4.1 GarminService

Odpowiada za komunikację z Garmin Connect przez pakiet `alexoid/laravel-garmin`:

- `importActivities()` – pobiera do 1000 aktywności i zapisuje je w bazie,
- `syncActivities()` – pobiera wyłącznie aktywności nowsze niż ostatnia zapisana.

Prędkości z Garmin (w m/s) są przeliczane na km/h przez mnożnik $\times 3,6$.

2.2.4.2 GpxParser

Parsuje pliki GPX (format XML) i wyodrębnia:

- listę punktów trasy (szerokość/długość geograficzna, wysokość, czas),
- całkowity dystans – obliczony wzorem Haversine’a,
- przewyższenie dodatnie (*elevation gain*),
- czas aktywności, datę, typ.

Dla wydajności trasa jest upraszczana do maksymalnie 500 punktów.

2.2.4.3 DashboardService

Agreguje dane dla dashboardu użytkownika:

- łączny dystans, łączny czas, średnia prędkość,
- lista ostatnich aktywności,
- dane szeregów czasowych dla wykresu.

2.2.4.4 ActivityLogger

Rejestruje akcje użytkownika w tabeli `activity_logs`. Zdefiniowane akcje: `GARMIN_SYNC_STARTED`, `GARMIN_SYNC_COMPLETED`, `ACTIVITY_CREATED`, `ACTIVITY_DELETED`.

2.2.5 Uwierzytelnianie

System uwierzytelniania oparty na **Laravel Breeze**:

- sesje przechowywane w bazie danych (tabela `sessions`),
- middleware `auth` chroni wszystkie trasy wymagające zalogowania,
- middleware `HandleInertiaRequests` eksponuje dane zalogowanego użytkownika (`auth.user`) do każdej strony Vue jako *shared props*.

2.3 Frontend

2.3.1 Vue 3 + TypeScript

Interfejs użytkownika napisany jest w Vue 3 z Composition API (`<script setup lang="ts">`). TypeScript działa w trybie *strict* – wszystkie zmienne są typowane, użycie `any` jest zakazane.

2.3.2 Kluczowe biblioteki frontendowe

Biblioteka	Zastosowanie
Reka UI	Beznakładowe prymitywy UI (Dialog, DropdownMenu, Table i in.)
Tailwind CSS v4	Utility-first CSS – stylowanie komponentów

Lucide Vue Next	Ikony wektorowe SVG
Vue i18n 11	Internacjonalizacja – klucze tłumaczeń w szablonach
Vue Sonner	Powiadomienia toast
Unovis	Wykresy liniowe (biblioteka wizualizacji danych)
Leaflet + OpenStreetMap	Interaktywna mapa tras GPS
TanStack Vue Table	Tabela danych z sortowaniem i filtrowaniem
Ziggy	Generowanie URL z nazwanymi trasami Laravel w JS
VueUse	Utilities Composition API (hooks pomocnicze)

2.3.3 Struktura plików frontendowych

```

1 resources/js/
2     Pages/           # Komponenty stron (1:1 z routami)
3         Activities/  # Widoki aktywności
4         Auth/        # Strony logowania/rejestracji
5         Dashboard/   # Komponenty dashboardu
6         Profile/     # Profil użytkownika
7     Components/
8         ui/          # Prymitywy Reka UI (nie edytować)
9     Layouts/         # AppLayout, GuestLayout
10    i18n/             # pl.ts, en.ts, index.ts
11    app.ts            # Punkt wejściowy aplikacji

```

Listing 2.2: Struktura katalogu resources/js/

2.3.4 Konwencje frontendowe

- Tylko `<script setup lang="ts">` – zakaz używania Options API,
- alias ścieżki `@/` mapuje na `resources/js/`,
- nawigacja przez komponent `<Link>` z Inertia (nie tagi `<a>`),
- tryb ciemny (dark mode) – każdy nowy komponent musi wspierać oba warianty,
- formatowanie przez Prettier (pojedyncze cudzysłowy, średniki, 80 znaków, 2 spacje wcięcia).

2.4 Środowisko deweloperskie

2.4.1 Docker / Laravel Sail

Środowisko lokalne opiera się na **Laravel Sail** – zestawie skryptów Docker skonfigurowanych dla Laravel. Sail uruchamia kontenery:

- **PHP/Laravel** – serwer aplikacji,
- **PostgreSQL 18** – baza danych,
- **Adminer** – graficzny interfejs bazy danych dostępny pod adresem `http://localhost:8080`,
- **Node.js** – kompilacja frontendu przez Vite.

2.4.2 Dev Container

Projekt skonfigurowany jest dla **VS Code Dev Containers** – jednorazowe otwarcie projektu w kontenerze zapewnia gotowe do pracy środowisko bez ręcznej instalacji zależności.

2.4.3 Haki pre-commit

Husky uruchamia automatycznie `npm lint-staged` przed każdym commitem:

- ESLint z auto-naprawą dla plików `.ts` / `.vue`,
- Prettier dla formatowania kodu,
- Laravel Pint dla PHP.

Pomijanie haków przez `-no-verify` jest zakazane.

Rozdział 3

Wdrożone zagadnienia kwalifikacyjne

Poniżej wyszczególnione zostały zagadnienia techniczne i architektoniczne wdrożone w projekcie, stanowiące o dojrzałości technicznej rozwiązania.

3.1 Architektura i wzorce projektowe

1. Wzorzec MVC z warstwą serwisów

Aplikacja stosuje wzorzec Model–Widok–Kontroler rozszerzony o dedykowaną warstwę serwisów. Kontrolery delegują całą logikę biznesową do klas serwisowych (`GarminService`, `DashboardService`, `GpxParser`), pozostając cienkimi adapterami między rutinami a logiką.

2. Architektura SPA z Inertia.js

Zamiast tradycyjnego REST API + osobna aplikacja SPA, projekt stosuje Inertia.js jako protokół łączący Laravel z Vue 3. Backend renderuje komponenty Vue z danymi bez konieczności budowania i utrzymywania API JSON. Nawigacja odbywa się bez przeładowania strony.

3. Dependency Injection

Serwisy wstrzykiwane są przez konstruktory kontrolerów za pośrednictwem kontenera IoC Laravel. Przykład: `DashboardController` deklaruje zależność od `DashboardService` przez typowanie konstruktora.

4. Repozytorium zdarzeń – Activity Logs

Zaimplementowano niezmienny rejestr zdarzeń (append-only log) w tabeli `activity_logs`, rejestrujący kluczowe akcje z sygnaturą czasową. Tabela nie posiada kolumny `updated_at` i nigdy nie jest aktualizowana – jedynie dopisywana.

3.2 Bezpieczeństwo

5. Szyfrowanie wrażliwych danych

Hasła do konta Garmin Connect przechowywane są w bazie danych z użyciem wbudowanego mechanizmu szyfrowania Laravel (cast `encrypted`). Klucz szyfrowania pochodzi z `APP_KEY`.

6. Haszowanie haseł

Hasła użytkowników haszowane są algorytmem bcrypt (12 rund, konfigurowane przez `BCRYPT_ROUNDS`), nigdy nie przechowywane w postaci jawnej.

7. Izolacja danych między użytkownikami

Każde zapytanie do tabeli `activities` scopowane jest do zalogowanego użytkownika (`WHERE user_id = ?`). Brak możliwości dostępu do cudzych aktywności.

8. Walidacja danych wejściowych

Dane formularzy walidowane są w dedykowanych klasach `Http/Requests/` przed dotarciem do kontrolerów. Walidacja po stronie serwera jest obowiązkowa – nie polegamy wyłącznie na walidacji frontendu.

9. Autoryzacja tras

Middleware `auth` chroni wszystkie trasy wymagające zalogowania. Trasy dostępne dla gości (rejestracja, logowanie) otoczone są middleware `guest`, uniemożliwiającym dostęp po zalogowaniu.

10. CSRF Protection

Laravel automatycznie weryfikuje tokeny CSRF dla wszystkich żądań zmieniających stan (`POST`, `PUT`, `PATCH`, `DELETE`). `Inertia.js` dołącza token do nagłówków żądań.

3.3 Integracje zewnętrzne

11. Integracja z Garmin Connect API

Wdrożono dwutrybową integrację z platformą Garmin Connect przez pakiet `alexoid/laravel-garmin-connect`. Pełny import historii oraz inkrementalna synchronizacja nowych aktywności. Przeliczanie jednostek ($\text{m/s} \rightarrow \text{km/h}$).

12. Parser plików GPX

Zaimplementowano własny parser plików GPS Exchange Format (XML). Parser oblicza dystans wzorem Haversine'a, wyznacza przewyższenie dodatnie i upraszcza trasę algorytmem Douglas-Peucker do 500 punktów dla wydajności.

13. Mapa interaktywna – Leaflet + OpenStreetMap

Trasy aktywności wizualizowane są na interaktywnej mapie z kafli OpenStreetMap,

bez konieczności korzystania z płatnych API map.

3.4 Frontend i UX

14. Tryb ciemny (Dark Mode)

Interfejs w pełni wspiera tryb ciemny Tailwind CSS. Każdy komponent definiuje warianty dla jasnego i ciemnego motywu.

15. Internacjonalizacja (i18n)

Aplikacja obsługuje dwa języki (polski i angielski) z użyciem Vue i18n. Żadny widoczny ciąg znaków nie jest hardkodowany w szablonach – wszystko przechodzi przez klucze tłumaczeń.

16. Powiadomienia Toast

System flash-wiadomości zintegrowany między backendem (Inertia flash) a frontendem (Vue Sonner). Backend ustawia komunikat, frontend wyświetla powiadomienie bez dodatkowego kodu po stronie stron.

17. Komponenty headless UI – Reka UI

Prymitywy interfejsu (okna dialogowe, menu rozwijane, tabele) zbudowane są na bazie biblioteki Reka UI (fork Radix Vue), zapewniającej pełną dostępność (a11y) i poprawne zarządzanie focusem klawiatury.

18. Wykresy danych – Unovis

Dashboard prezentuje wykres liniowy aktywności z użyciem biblioteki Unovis, dedykowanej do wizualizacji danych w aplikacjach Vue.

3.5 Jakość kodu i DevOps

19. TypeScript Strict Mode

Cały frontend pisany jest w TypeScript z włączonym trybem *strict* (`vue-tsc` jako sprawdzanie typów). Użycie `any` jest zakazane.

20. Automatyczne formatowanie i lintowanie

Pipeline jakości kodu: Prettier (formatowanie), ESLint (lintowanie TypeScript/Vue), Laravel Pint (formatowanie PHP). Haki Husky uruchamiają `lint-staged` automatycznie przed każdym commitem.

21. Konteneryzacja – Docker / Laravel Sail

Środowisko deweloperskie w pełni skonteneryzowane, uruchamiane jedną komendą. VS Code Dev Containers pozwala na natychmiastowe uruchomienie projektu bez instalacji zależności na hoście.

22. Migracje bazy danych

Każda zmiana schematu bazy to osobny, wersjonowany plik migracji Laravel. Historia zmian schematu jest częścią historii Git i pozwala na odtworzenie dowolnego stanu bazy.

23. Graficzny interfejs bazy danych – Adminer

Do środowiska deweloperskiego dołączony jest kontener **Adminer** – webowy klient PostgreSQL dostępny pod adresem `http://localhost:8080`. Narzędzie umożliwia przeglądanie i edycję danych, podgląd struktury tabel oraz wykonywanie zapytań SQL bez konieczności korzystania z wiersza poleceń. Kontener skonfigurowany jest z domyślnym serwerem `pgsql` i ciemnym motywem wizualnym.

24. Kolejowanie zadań

Synchronizacja z Garmin wykonywana jest za pośrednictwem kolejki zadań Laravel (driver: `database`), co pozwala na asynchroniczne wykonywanie długotrwałych operacji bez blokowania odpowiedzi HTTP.

25. Leniwy zapis danych trasy GPS (lazy persistence)

Dane trasy GPS dla aktywności Garmin pobierane są z API Garmin tylko przy pierwszym wyświetleniu mapy, a następnie trwale zapisywane w tabeli `activity_gpx`. Każde kolejne otwarcie mapy korzysta z bazy danych – bez ponownego zapytania do API Garmin. Tokeny OAuth Garmin przechowywane są osobno w cache Laravel (`LaravelCacheTokenStore`).

Rozdział 4

Instrukcja uruchomienia systemu

4.1 Wymagania wstępne

4.1.1 Uruchomienie lokalne (Dev Container)

- **Docker Desktop** – wersja 4.0 lub nowsza,
- **Visual Studio Code** z rozszerzeniem *Dev Containers* (`ms-vscode-remote.remote-containers`)
- konto Garmin Connect (opcjonalnie, wymagane do integracji Garmin).

4.1.2 Uruchomienie zdalne / produkcyjne

- serwer Linux z PHP 8.2+, Node.js 20+, Composer, PostgreSQL 18,
- lub środowisko Docker z możliwością uruchomienia `docker-compose`.

4.2 Uruchomienie lokalne (Dev Container – zalecane)

4.2.1 Krok 1: Sklonowanie repozytorium

```
git clone https://github.com/Console-dungeon/strava-clone.git
cd strava-clone
```

4.2.2 Krok 2: Inicjalizacja środowiska

```
make init
```

Komenda `make init` wykonuje:

1. kopiuje `.env.example` → `.env`,
2. instaluje zależności PHP przez Composer (`composer install`).

4.2.3 Krok 3: Otwarcie w kontenerze

W VS Code: **Ctrl+Shift+P** → *Dev Containers: Reopen in Container*.

VS Code automatycznie:

1. zbuduje kontenery Docker (PHP, PostgreSQL, Node.js),
2. zainstaluje zależności Node.js,
3. skonfiguruje środowisko.

4.2.4 Krok 4: Migracja bazy danych

```
php artisan migrate
```

4.2.5 Krok 5: Uruchomienie serwera deweloperskiego

```
composer dev
```

Komenda uruchamia równolegle (za pomocą `concurrently`):

- `php artisan serve` – serwer PHP na porcie 8000,
- `php artisan queue:listen` – worker kolejki,
- `pnpm run dev` – serwer Vite (hot-reload frontendu).

Aplikacja dostępna pod adresem: `http://localhost:8000`

4.2.6 Alternatywne uruchomienie przez F5

Projekt skonfigurowany jest dla debuggera VS Code. Naciśnięcie klawisza **F5** uruchamia serwer deweloperski z aktywnym debuggerem PHP.

4.3 Konfiguracja integracji Garmin (opcjonalne)

1. Zaloguj się do aplikacji i przejdź do strony **Profil**.
2. W sekcji *Garmin Connect* wpisz adres e-mail i hasło konta Garmin.
3. Kliknij *Zapisz* – dane zostaną zaszyfrowane i zapisane w bazie.
4. Na dashboardzie użyj przycisku **Importuj z Garmin** (jednorazowy pełny import) lub **Synchronizuj** (tylko nowe aktywności).

4.4 Uruchomienie zdalne / produkcyjne

4.4.1 Krok 1: Przygotowanie serwera

Upewnij się, że na serwerze zainstalowane są:

```
php --version      # >= 8.2
node --version     # >= 20
composer --version
psql --version     # PostgreSQL >= 15
```

4.4.2 Krok 2: Sklonowanie i konfiguracja

```
git clone https://github.com/Console-dungeon/strava-clone.git
cd strava-clone
cp .env.example .env
```

4.4.3 Krok 3: Konfiguracja zmiennych środowiskowych

Edytuj plik `.env` i ustaw:

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://twoja-domena.pl

DB_CONNECTION=pgsql
DB_HOST=localhost
DB_PORT=5432
DB_DATABASE=strava_clone
DB_USERNAME=uzyskownik
DB_PASSWORD=twoje_haslo

QUEUE_CONNECTION=database
SESSION_DRIVER=database
CACHE_STORE=database
```

4.4.4 Krok 4: Instalacja zależności i build

```
# Zale no ci PHP (bez deweloperskich)
composer install --no-dev --optimize-autoloader

# Wygenerowanie klucza aplikacji
```

```
php artisan key:generate

# Zale no ci Node.js i build frontendu
npm install
npm run build

# Migracja bazy danych
php artisan migrate --force

# Optymalizacja konfiguracji
php artisan config:cache
php artisan route:cache
php artisan view:cache
```

4.4.5 Krok 5: Konfiguracja serwera WWW

Nginx (przykładowa konfiguracja):

```
server {
    listen 80;
    server_name twoja-domena.pl;
    root /var/www/strava-clone/public;
    index index.php;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ \.php$ {
        fastcgi_pass unix:/run/php/php8.2-fpm.sock;
        fastcgi_param SCRIPT_FILENAME
$realpath_root$fastcgi_script_name;
        include fastcgi_params;
    }
}
```

4.4.6 Krok 6: Uruchomienie workera kolejki

Worker kolejki jest wymagany do działania synchronizacji Garmin:

```
# Uruchomienie przez Supervisor (zalecane na produkcji)
php artisan queue:work --sleep=3 --tries=3
```

```
# Lub jako proces systemd / supervisor
```

4.4.7 Krok 7: Uprawnienia katalogów

```
chmod -R 775 storage bootstrap/cache  
chown -R www-data:www-data storage bootstrap/cache
```

4.5 Przydatne komendy

Komenda	Działanie
composer dev	Uruchomienie serwera deweloperskiego
php artisan migrate	Wykonanie migracji bazy danych
php artisan migrate:rollback	Cofnięcie ostatniej migracji
pnpm lint	Lintowanie kodu TypeScript/Vue
pnpm format	Formatowanie kodu Prettier
./vendor/bin/pint	Formatowanie kodu PHP
pnpm build	Build produkcyjny frontendu
php artisan queue:listen	Nasłuchiwanie na zadania kolejki
php artisan tinker	Interaktywna konsola Laravel

Rozdział 5

Wnioski projektowe

5.1 Podsumowanie realizacji projektu

Projekt zakończył się zrealizowaniem kompletnej aplikacji webowej do śledzenia aktywności fizycznych. W toku pracy zespołowej zdobyto szereg doświadczeń i obserwacji dotyczących organizacji pracy, doboru narzędzi i metodyki wytwarzania oprogramowania. Poniżej przedstawiono wnioski, które mogą być wartościowe przy realizacji kolejnych projektów podobnej skali.

5.2 Kluczowe wnioski

5.2.1 Bezpośrednia komunikacja zespołu jako fundament projektu

Jednym z najistotniejszych czynników sukcesu projektu okazały się regularne spotkania członków zespołu – zarówno osobiste, jak i przez wideo. Omawianie bieżących kwestii w czasie rzeczywistym, nawet w formie krótkich sesji online, pozwalało na szybkie uzgodnienie kierunku implementacji, rozwiązywanie nieporozumień co do architektury i podziału zadań oraz uniknięcie sytuacji, w których dwóch programistów pracuje nad tym samym problemem lub – co gorsza – w sprzecznych kierunkach.

Próby koordynacji wyłącznie przez komentarze w kodzie, wiadomości tekstowe czy opis zadań w systemie kontroli wersji okazały się niewystarczające przy bardziej złożonych decyzjach projektowych. Bezpośrednia komunikacja, choćby krótka, pozostaje niezastąpiona i praktycznie niezbędna przy omawianiu kluczowych kwestii projektu i wdrażanych rozwiązań.

5.2.2 Iteracyjna praca z małymi przyrostami

Projekt prowadzony był metodą iteracyjną – zamiast próby zaimplementowania wszystkich funkcjonalności naraz, pracowano nad małymi, samodzielnymi przyrostami. Każda iteracja kończyła się działającą funkcją dostępną w głównej gałęzi projektu. Takie podejście przyniosło kilka wymiernych korzyści:

- Błędy pojawiały się w kontekście świeżo dodanego, małego fragmentu kodu – znacznie łatwiej było zidentyfikować ich źródło i naprawić je, zanim zaciemniły większy fragment systemu.
- Historia Git pozostawała czytelna – każdy commit odpowiadał konkretnej, skończonej funkcji.
- Projekt był stale w stanie działającym, co eliminowało ryzyko „dużego wybuchu” na etapie integracji.

Regularne, małe wdrożenia minimalizowały ryzyko i ułatwiały naprawę błędów. Podejście monolityczne (duże zmiany rzadko) wielokrotnie prowadziłyby do trudnych do debugowania konfliktów.

5.2.3 Praca zespołowa jako akcelerator skali i jakości

Realizacja projektu w zespole pozwoliła na zbudowanie aplikacji o znacznie szerszym zakresie funkcjonalnym, niż byłoby to możliwe w tym samym czasie w pracy indywidualnej. Podział pracy według kompetencji i zainteresowań – jeden programista skupia się na backendzie i integracji Garmin, inny na fronendzie i wizualizacji – pozwolił na równoległe postępy w różnych obszarach systemu. Wzajemna weryfikacja kodu i wspólne rozwiązywanie problemów prowadziły do powstawania rozwiązań wyższej jakości niż praca w izolacji.

Praca zespołowa wymaga inwestycji w komunikację i koordynację, jednak zwrot z tej inwestycji jest wyraźny zarówno w tempie powstawania nowych funkcjonalności, jak i w końcowej jakości produktu.

5.2.4 Agenci AI jako katalizator produktywności

Projekt korzystał z narzędzi opartych na agentach sztucznej inteligencji (m.in. Claude Code) do wspomagania implementacji. Zastosowanie agentów AI przyniosło zauważalne skrócenie czasu potrzebnego na tworzenie nowych funkcjonalności – szkielety kodu, serwisy, komponenty Vue czy migracje bazy danych powstawały w ułamku czasu, jaki zajęłoby ich ręczne napisanie od zera.

Agenci sprawdzali się szczególnie dobrze przy:

- generowaniu powtarzalnych struktur (migracje, kontrolery, komponenty),
- eksploracji nieznanymi bibliotek i ich API,
- szybkim prototypowaniu rozwiązań do weryfikacji koncepcji,
- refaktoryzacji i poprawie jakości istniejącego kodu.

Kluczowym wnioskiem jest jednak to, że agenci AI są narzędziem amplifikującym – zwiększają produktywność doświadczonego programisty, który rozumie generowany kod i jest w stanie go zweryfikować, a nie zastępują samodzielne myślenie projektowe.

5.3 Rekomendacje na przyszłość

Na podstawie doświadczeń zdobytych podczas realizacji projektu sformułowano następujące rekomendacje dla przyszłych przedsięwzięć:

1. **Ustalenie kanałów komunikacji na początku projektu** – jasno określić, kiedy używamy wiadomości tekstowej, a kiedy planujemy spotkanie.
2. **Zdefiniowanie standardów kodu przed pierwszym commitem** – konfiguracja linterów, formaterów i haków pre-commit od pierwszego dnia eliminuje późniejsze konflikty stylistyczne.
3. **Małe, częste commity zamiast rzadkich dużych paczek zmian** – ułatwia debugowanie, code review i cofanie błędnych decyzji.
4. **Integracja narzędzi AI od początku projektu** – wcześniejsze oswojenie się z narzędziami AI w kontekście projektu przynosi większe korzyści niż sięganie po nie dopiero przy trudniejszych zadaniach.

5.4 Zakończenie

Projekt pozwolił zbudować kompletną aplikację do śledzenia aktywności fizycznych z integracją zewnętrznych platform sportowych, nowoczesnym frontendem i przemyślaną architekturą backendową. Sukces projektu jest wynikiem połączenia regularnej komunikacji w zespole, iteracyjnego podejścia do wytwarzania oprogramowania, efektywnej współpracy i umiejętnego wykorzystania nowoczesnych narzędzi wspomagania programowania. Doświadczenia zdobyte podczas realizacji projektu stanowią solidną podstawę dla kolejnych, bardziej ambitnych przedsięwzięć programistycznych.